

TechConnect

Arquitetura, Padrões de Projeto, Documentação de API e Inteligência Artificial

Curso Análise e Desenvolvimento de Sistemas (UNINOVE)
Identificação Grupo 6

INTEGRANTES OFICIAIS

Lucas Alves — RA: 924111565
Diogo Antunes — RA: 924110067
Daniel Augusto — RA: 924110360
Lucas Rufo — RA: 924111778
Felipe Augusto — RA: 924107836
Isaque Cândido — RA: 924111157

2. Padrões de Projeto (GoF)

Para garantir extensibilidade e o correto desacoplamento na arquitetura da API Spring Boot do TechConnect, implementamos três padrões arquiteturais distintos.

Padrão 1: Factory Method CRIACIONAL

Problema Resolvido: o sistema do TechConnect precisa lidar com múltiplos processadores de pagamento (Pix, Cartão, Boleto). Instanciar essas classes de integração de forma rígida polui a camada de serviço. A Factory encapsula a lógica de criação do processador correto com base na string do método enviada pelo cliente.

DIAGRAMA UML (CÓDIGO MERMAID)

```
classDiagram
    class PaymentProcessor {
        <<interface>>
        +process(amount: double) void
    }
    class PixProcessor {
        +process(amount: double) void
    }
    class CardProcessor {
        +process(amount: double) void
    }
    class PaymentProcessorFactory {
        +createProcessor(method: String) PaymentProcessor
    }
    PaymentProcessor <|-- PixProcessor
    PaymentProcessor <|-- CardProcessor
    PaymentProcessorFactory ..> PaymentProcessor : Instancia
```

CÓDIGO DE APLICAÇÃO (JAVA / SPRING)

```
public interface PaymentProcessor {
    void process(BigDecimal amount);
}

@Component("pix")
public class PixProcessor implements PaymentProcessor {
    public void process(BigDecimal amount) { /* Integração com o Gateway via PIX */ }
}

@Component
public class PaymentProcessorFactory {
    @Autowired
    private Map<String, PaymentProcessor> processors;

    public PaymentProcessor createProcessor(String method) {
        PaymentProcessor processor = processors.get(method.toLowerCase());
        if (processor == null) throw new IllegalArgumentException("Método inválido");
        return processor;
    }
}
```

Padrão 2: Strategy COMPORTAMENTAL

Problema Resolvido: o cálculo do preço final cobrado por um atendimento técnico pode variar dinamicamente. Técnicos comuns cobram apenas o valor base (`hourly_rate`); técnicos Premium aplicam taxas de urgência ou a plataforma cobra tarifas de intermediação flutuantes. O Strategy isola esses algoritmos, removendo condicionais `if/else` complexas do sistema de faturamento.

DIAGRAMA UML (CÓDIGO MERMAID)

```

classDiagram
    class PricingStrategy {
        <<interface>>
        +calculate(hourlyRate: double, hours: int) double
    }
    class StandardPricing {
        +calculate(hourlyRate: double, hours: int) double
    }
    class PremiumPricing {
        +calculate(hourlyRate: double, hours: int) double
    }
    class ServiceBillingContext {
        -PricingStrategy strategy
        +executeCalculation(rate, hrs) double
    }
    PricingStrategy <|.. StandardPricing
    PricingStrategy <|.. PremiumPricing
    ServiceBillingContext --> PricingStrategy

```

CÓDIGO DE APLICAÇÃO (JAVA)

```

public interface PricingStrategy {
    BigDecimal calculate(BigDecimal hourlyRate, int hours);
}

public class StandardPricing implements PricingStrategy {
    public BigDecimal calculate(BigDecimal hourlyRate, int hours) {
        return hourlyRate.multiply(new BigDecimal(hours));
    }
}

public class PremiumPricing implements PricingStrategy {
    public BigDecimal calculate(BigDecimal hourlyRate, int hours) {
        BigDecimal base = hourlyRate.multiply(new BigDecimal(hours));
        return base.multiply(new BigDecimal("1.15")); // Taxa adicional de 15%
    }
}

```

Padrão 3: Observer COMPORTAMENTAL

Problema Resolvido: quando o status de uma atividade é modificado na plataforma, múltiplos ecossistemas do backend precisam reagir de forma assíncrona (o ecossistema de e-mail dispara alertas, logs no MongoDB são gerados e estatísticas no cache Redis são limpas). O padrão implementado via Spring Events desacopla o emissor das ações secundárias.

DIAGRAMA UML (CÓDIGO MERMAID)

```

classDiagram
    class ApplicationEventPublisher {
        +publishEvent(event)
    }
    class ServiceStatusEvent {
        +serviceId: Long
        +status: String
    }
    class EmailNotificationListener {
        +onApplicationEvent(event)
    }
    class AuditLogListener {
        +onApplicationEvent(event)
    }
    ApplicationEventPublisher ..> ServiceStatusEvent : Publica
    ServiceStatusEvent <-- EmailNotificationListener : Observa
    ServiceStatusEvent <-- AuditLogListener : Observa

```

CÓDIGO DE APLICAÇÃO (JAVA / SPRING BOOT)

```
public class ServiceStatusEvent {
    private Long serviceId;
    private String status;
    // construtor e getters
}

@Component
public class EmailListener {
    @EventListener
    @Async // Executa em paralelo de forma assíncrona
    public void handleStatusChange(ServiceStatusEvent event) {
        // Aciona o serviço de e-mail externo para notificar as partes
    }
}
```

3. Documentação de APIs

A API REST do TechConnect encontra-se mapeada no ecossistema OpenAPI 3.0 e pode ser visualizada em tempo de execução via Swagger.

Link da Documentação (SwaggerHub): <https://app.swaggerhub.com/apis/aaa-5f9/techconnect-core-api/1.0.0>

Mapeamento Técnico dos Endpoints do Código Real

Nossa aplicação expõe 15 endpoints funcionais derivados diretamente do código-fonte dos controladores desenvolvidos:

Método	Endpoint	Categoria	Descrição
POST	/api/auth/cadastrar	Autenticação	Registrar nova conta de usuário
POST	/api/auth/login	Autenticação	Autenticar usuário e emitir token JWT
GET	/api/avaliacoes/tecnico/{tecnicoid}	Avaliações	Listar todas as avaliações recebidas por um técnico
POST	/api/avaliacoes	Avaliações	Emitir nova avaliação com nota de 1 a 5
GET	/api/favoritos	Favoritos	Listar técnicos marcados como favoritos por um cliente
POST	/api/favoritos	Favoritos	Adicionar profissional à lista de favoritos
DELETE	/api/favoritos	Favoritos	Remover profissional da lista de favoritos
GET	/api/mensagens	Mensagens - Chat	Recuperar histórico de mensagens entre duas partes
POST	/api/mensagens	Mensagens - Chat	Enviar mensagem de texto em tempo real no chat
GET	/api/tecnicos	Técnicos	Retornar todos os profissionais habilitados
POST	/api/tecnicos	Técnicos	Cadastrar perfil profissional e taxa por hora
GET	/api/tecnicos/buscar	Técnicos	Filtrar técnicos por competências (skills)
GET	/api/usuarios/{id}	Utilizadores	Obter informações cadastrais base de um perfil
PUT	/api/usuarios/{id}	Utilizadores	Atualizar dados do usuário
PATCH	/api/usuarios/{id}/foto	Utilizadores	Atualizar endereço ou avatar da foto de perfil

ESPECIFICAÇÃO OPENAPI 3.0 (YAML)

```
openapi: 3.0.3
info:
  title: TechConnect Core API
  version: 1.0.0
paths:
  /api/auth/cadastrar:
    post:
      tags: [Autenticação]
      summary: Registrar nova conta de usuário
      requestBody:
        content:
          application/json:
            schema:
              properties:
                nome: { type: string, example: "Ricardo Silva" }
                email: { type: string, example: "ricardo@email.com" }
                senha: { type: string, example: "senha123" }
                role: { type: string, example: "client" }
      responses:
        '200': { description: Cadastro concluído }
  /api/auth/login:
    post:
      tags: [Autenticação]
      summary: Autenticar usuário e emitir token JWT
      responses:
        '200': { description: Login efetuado. Retorna token e ID }
  /api/avaliacoes/tecnico/{tecnicoId}:
    get:
      tags: [Avaliações]
      summary: Listar todas as avaliações recebidas por um técnico
  /api/avaliacoes:
    post:
      tags: [Avaliações]
      summary: Emitir nova avaliação com nota de 1 a 5
  /api/favoritos:
    get:
      tags: [Favoritos]
      summary: Listar técnicos marcados como favoritos por um cliente
    post:
      tags: [Favoritos]
      summary: Adicionar profissional à lista de favoritos
    delete:
      tags: [Favoritos]
      summary: Remover profissional da lista de favoritos
  /api/mensagens:
    get:
      tags: [Mensagens - Chat]
      summary: Recuperar histórico de mensagens entre duas partes
    post:
      tags: [Mensagens - Chat]
      summary: Enviar mensagem de texto em tempo real no chat
  /api/tecnicos:
    get:
      tags: [Técnicos]
      summary: Retornar todos os profissionais habilitados
    post:
      tags: [Técnicos]
      summary: Cadastrar perfil profissional e taxa por hora
  /api/tecnicos/buscar:
    get:
      tags: [Técnicos]
      summary: Filtrar técnicos por competências (skills)
  /api/usuarios/{id}:
    get:
      tags: [Utilizadores]
      summary: Obter informações cadastrais base de um perfil
    put:
      tags: [Utilizadores]
      summary: Atualizar dados do usuário
  /api/usuarios/{id}/foto:
    patch:
      tags: [Utilizadores]
      summary: Atualizar endereço ou avatar da foto de perfil
```

4. Inteligência Artificial na Aplicação

Descrição do Caso de Uso: Triagem e Classificação de Incidentes de TI

Para resolver o problema de clientes que descrevem os incidentes de TI de forma muito genérica (ex.: "Meu computador não liga e está apitando"), incluímos um módulo de IA no fluxo de triagem. A IA analisa o texto fornecido pelo cliente e executa duas tarefas essenciais:

- Classificação Categórica:** categoriza o incidente de forma padronizada (Hardware, Software, Redes, Infraestrutura).
- Mapeamento de Competências:** extrai os requisitos técnicos necessários e sugere palavras-chave alinhadas à nossa tabela relacional de `skills`. O sistema consome esses termos para fazer uma busca dinâmica e precisa, priorizando profissionais aptos na plataforma.

Ferramentas e Justificativa

API Escolhida: Gemini 1.5 Flash (via SDK oficial do Google Cloud / LangChain4j).

Justificativa: trata-se do modelo com menor tempo de resposta (latência) para processamento em tempo real na web, possui suporte robusto e nativo a saídas em formato JSON estruturado (Structured Outputs) e apresenta baixo custo operacional de tokens.

Prova de Conceito (PoC) da Funcionalidade de IA

Abaixo está o componente funcional em execução na nossa API Spring Boot, estruturando a chamada de inferência e tratando a resposta via objetos de negócio de forma assíncrona.

CÓDIGO DE APLICAÇÃO (JAVA)

```
@Service
public class TechConnectAiService {

    @Value("${google.ai.api.key}")
    private String apiKey;

    public AiAnalysisResult analyzeIncident(String customerText) {
        // Definição do comportamento estrito esperado para a saída
        String systemInstruction = "Você é um especialista em suporte de TI. "
            + "Analise o incidente informado pelo usuário e retorne unicamente o formato JSON: "
            + "{ \"category\": \"string\", \"suggestedSkills\": [\"string\"] }";

        // Execução do payload via HTTP Client encapsulado do SDK Gemini
        String jsonResponse = GeminiApiClient.builder()
            .apiKey(this.apiKey)
            .model("gemini-1.5-flash")
            .systemInstruction(systemInstruction)
            .prompt("Analise o seguinte problema relatado: " + customerText)
            .execute();

        // Parse dinâmico do JSON string para Objeto de Negócio Java
        return ObjectMapper.readValue(jsonResponse, AiAnalysisResult.class);
    }
}
```

Exemplo de Entrada (Usuário):

"Minha impressora parou de funcionar na rede, os computadores tentam enviar o documento mas dá erro de conexão de IP."

SAÍDA ESTRUTURADA GERADA PELA IA (JSON)

```
{
  "category": "REDES_PERIFERICOS",
  "suggestedSkills": ["Instalação de Impressoras", "Configuração de Rede Local", "Troubleshooting IP"]
}
```

5. Checkpoint 2 — Estado Atual do Projeto

Progresso Geral do Projeto

✓ Concluído

- Modelagem relacional em MySQL com todas as restrições de integridade estruturadas.
- Configuração do cache elástico e do mecanismo Sorted Set para Rankings usando Redis.
- Codificação de todos os 6 controladores (Controllers) de negócio.
- Documentação e mapeamento da arquitetura OpenAPI 3.0 via Swagger.
- Integração das telas do Frontend React SPA com os endpoints autenticados da API.

⌚ Em Andamento / Próximos Passos

- Solicitar orçamento online para estimativa de custos.
- Acompanhar o status do serviço.
- Assinar plano premium SaaS para destaque (Técnico).

Links de Acesso Obrigatórios

Repositório GitHub Atualizado: <https://github.com/DiogoAntunes0/tech-connect.git>
Quadro Kanban (GitHub Projects): <https://github.com/users/DiogoAntunes0/projects/2>